TDEW Estimation: PRAM Pseudocode and Complexity

Bucket-year anomaly local linear regression pipeline

Piero Palacios

2026-05-07

Table of contents

1 Purpose

This document describes the full algorithm used to estimate daily dewpoint temperature (`TD`) from minimum temperature (`TMIN`) for many spatial locations (`ID`). The design follows a PRAM-style view: independent operations are marked with `pardo`.

The project uses an anomaly local linear regression model:

$$TD_{anom}(t) = TD(t) - TD_{clim}(ID, doy)$$

$$TMIN_{anom}(t) = TMIN(t) - TMIN_{clim}(ID, doy)$$

For each spatial `ID` and each day-of-year `doy`, the model fits a weighted linear regression using `TMIN` anomaly and lagged anomaly variables.

2 PRAM Model

We use a CREW PRAM interpretation:

- Concurrent read: many processors may read shared input tables.
- Exclusive write: each processor writes to a disjoint output bucket, ID, or day-of-year result.
- Reductions, such as sums and means, are treated as parallel reductions.
- `pardo` means the loop body can be executed in parallel.

The bucket-year design is important because it prevents 300k independent ID tasks from repeatedly scanning the same historical parquet files.

3 Symbols

Symbol	Meaning
`N`	Number of spatial locations, approximately 300,000 IDs
`Y`	Number of training years, approximately 40
`D`	Days per year, at most 366
`T`	Total days per ID, `T = Y * D`
`R`	Total rows per climate variable, `R = N * T`
`B`	Number of ID buckets, for example 1024
`p`	Number of available processors or worker slots
`h`	Local DOY half-window, default `h = 11`
`W`	Samples per local regression, `W = O(Y * h)`
`F`	Number of regression features plus intercept, constant here
`Q`	Number of failed DOYs to repair, `Q <= D`
`H`	Forecast horizon in days

In this project, `D`, `h`, and `F` are small constants in practice, but they are shown explicitly in the complexity formulas.

4 Algorithm 1: Full TDEW Pipeline

```
Algorithm TDEW-PIPELINE
Input:
  TD monthly files for all IDs and dates
  TMIN monthly files for all IDs and dates
  Grid table containing the expected IDs
  Parameters: B buckets, h local DOY window, p processors

Output:
  Daily climatology per (ID, doy)
  Bucket-year merged training dataset
  Regression coefficients per (ID, doy)
  Optional repair coefficients for incomplete DOYs
  Optional TD predictions

1  climatology <- BUILD-CLIMATOLOGY(TD, TMIN)
2  bucketed_training <- BUILD-BUCKET-YEAR-DATASET(TD, TMIN, B)
3  bucketed_climatology <- SHARD-CLIMATOLOGY(climatology, B)
4  coefficients <- TRAIN-COEFFICIENTS(bucketed_training,
                                      bucketed_climatology,
                                      h,
                                      B)
5  incomplete_doys <- CHECK-COMPLETENESS(coefficients, Grid)
6  if incomplete_doys is not empty then
```

```
7      patch <- RERUN-FAILED-DOYS(bucketed_training,
                                  bucketed_climatology,
                                  incomplete_doys,
                                  h,
                                  B)
8      coefficients <- PATCH-COEFFICIENTS(coefficients,
                                          patch,
                                          incomplete_doys)
9      incomplete_doys <- CHECK-COMPLETENESS(coefficients, Grid)
10 end if
11 predictions <- optional FORECAST-TD(coefficients, climatology, TMIN future)
12 return coefficients, predictions
```

5 Algorithm 2: Build Climatology

The climatology is the historical mean for each `(ID, doy)`.

```
Algorithm BUILD-CLIMATOLOGY
Input:
  TD rows:   (ID, date, TD)
  TMIN rows: (ID, date, TMIN)

Output:
  climatology rows: (ID, doy, TD_clim, TMIN_clim)

1  for each climate variable V in {TD, TMIN} pardo do
2      for each monthly file m of V pardo do
3          read rows from file m
4          for each row r pardo do
5              r.doy <- day_of_year(r.date)
6              emit partial record (r.ID, r.doy, r.value)
7          end for
8      end for
9      for each key (ID, doy) pardo do
10         sum_V(ID, doy) <- parallel sum of values with this key
11         count_V(ID, doy) <- parallel count of values with this key
12         clim_V(ID, doy) <- sum_V(ID, doy) / count_V(ID, doy)
13     end for
14 end for
15 join TD_clim and TMIN_clim by (ID, doy)
16 return climatology
```

Complexity:

$$W_{clim} = O(R)$$

$$T_p = O(R/p + \log R)$$

Space:

$$S_{clim} = O(ND)$$

6 Algorithm 3: Build Bucket-Year Training Dataset

This is the main redesign. The expensive monthly TD/TMIN merge is paid once. Rows are then distributed to deterministic ID buckets and compacted by year.

```
Algorithm BUILD-BUCKET-YEAR-DATASET
Input:
  TD monthly files
  TMIN monthly files
  Number of buckets B

Output:
  bucket-year training files:
  bucket b, year y -> rows (ID, date, TD, TMIN, doy)

1  for each year y pardo do
2      for each month m in year y pardo do
3          TD_m <- read TD rows for month m
4          TMIN_m <- read TMIN rows for month m
5          M_m <- hash join TD_m and TMIN_m by (ID, date)
6          for each row r in M_m pardo do
7              r.doy <- day_of_year(r.date)
8              r.bucket <- r.ID mod B
9              emit r to temporary partition (bucket = r.bucket, year = y)
10         end for
11     end for
12     for each bucket b in 0..B-1 pardo do
13         read all temporary rows for (bucket b, year y)
14         sort rows by (ID, date)
15         write compact file for (bucket b, year y)
16     end for
17 end for
18 return bucket-year training dataset
```

Complexity with hash joins:

$$W_{bucket} = O(R)$$

If sorting inside each bucket-year shard is included:

$$W_{bucket-sort} = O\left(R + \sum_{b,y} n_{b,y} \log n_{b,y}\right)$$

With balanced buckets, `n_{b,y} approx (N/B)D`, so:

$$W_{bucket-sort} = O\left(R + BY \cdot \frac{ND}{B} \log\left(\frac{ND}{B}\right)\right) = O\left(R \log\left(\frac{ND}{B}\right)\right)$$

Ideal parallel time:

$$T_p = O\left(\frac{R}{p} + \log\left(\frac{ND}{B}\right)\right)$$

Peak memory per bucket-year compaction:

$$S_{bucket-year} = O\left(\frac{ND}{B}\right)$$

7 Algorithm 4: Shard Climatology by Bucket

```
Algorithm SHARD-CLIMATOLOGY
Input:
  climatology rows (ID, doy, TD_clim, TMIN_clim)
  Number of buckets B

Output:
  bucketed climatology files

1  for each climatology row c pardo do
2      c.bucket <- c.ID mod B
3      emit c to bucket c.bucket
4  end for
5  for each bucket b in 0..B-1 pardo do
6      write climatology rows for bucket b
7  end for
8  return bucketed climatology
```

Complexity:

$$W_{clim-shard} = O(ND)$$

$$T_p = O(ND/p + \log B)$$

Peak memory with streaming:

$$S_{clim-shard} = O(ND/B)$$

If the full climatology is loaded at once, peak memory becomes `O(ND)`.

8 Algorithm 5: Train Anomaly Coefficients

For each `ID` and each target `doy`, fit a weighted regression over a circular day-of-year neighborhood.

```
Algorithm TRAIN-COEFFICIENTS
Input:
  bucket-year training dataset
  bucketed climatology
  Local DOY half-window h
  Number of buckets B

Output:
  coefficients per (ID, doy)

1  for each bucket b in 0..B-1 pardo do
2      train_b <- read all bucket-year training rows for bucket b
3      clim_b <- read climatology rows for bucket b
4      for each ID i in bucket b pardo do
5          X_i <- rows of train_b for ID i
```

```
        C_i <- rows of clim_b for ID i
        sort X_i by date
        for each row t in X_i pardo do
            TD_anom(t) <- TD(t) - TD_clim(i, doy(t))
            TMIN_anom(t) <- TMIN(t) - TMIN_clim(i, doy(t))
        end for
        for each row t in X_i pardo do
            create lag features:
              TD_anom_lag1(t), TD_anom_lag2(t), TMIN_anom_lag1(t)
        end for
        for each target day d in 1..366 pardo do
            N_d <- rows where circular_distance(doy(t), d) <= h
            remove rows in N_d with missing target or missing features
            if size(N_d) >= minimum_sample_count then
                for each row t in N_d pardo do
                    weight(t) <- tricube(circular_distance(doy(t), d) / h)
                end for
                beta <- weighted_least_squares(N_d)
                write coefficients for (ID i, doy d)
            end if
        end for
    end for
    write coefficient file for bucket b
    write failure log for bucket b, if needed
end for
return coefficient dataset
```

Per local regression:

$$W_{one-fit} = O(WF^2 + F^3)$$

Here `F` is constant, so:

$$W_{one-fit} = O(W) = O(Yh)$$

All coefficient training:

$$W_{train} = O(NDW) = O(NDYh)$$

Ideal PRAM span if buckets, IDs, and DOYs are fully parallel:

$$T_\infty = O(\log W + F^3)$$

With `p` processors:

$$T_p = O(NDYh/p + \log(Yh))$$

Practical bucket-parallel execution has at most `B` outer tasks, so:

$$T_p = O\left(\frac{NDYh}{\min(p, B)} + \text{I/O overhead}\right)$$

Peak memory per bucket task:

$$S_{train-bucket} = O\left(\frac{NT}{B} + \frac{ND}{B}\right)$$

The first term is the bucket training time series. The second term is the bucket climatology.

9 Algorithm 6: Check Completeness

```
Algorithm CHECK-COMPLETENESS
Input:
  coefficients
  expected ID count from grid

Output:
  incomplete DOYs

1  expected_count <- number of IDs in grid
2  for each coefficient row c pardo do
3      emit key c.doy with value c.ID
4  end for
5  for each doy d in 1..366 pardo do
6      observed_count(d) <- distinct count of IDs emitted for d
7      if observed_count(d) < expected_count then
8          mark d as incomplete
9      end if
10 end for
11 return incomplete DOYs
```

Let `C = ND` be the complete number of coefficient rows.

$$W_{check} = O(C)$$

$$T_p = O(C/p + \log C)$$

Space:

$$S_{check} = O(C)$$

The output itself is small: at most `D` incomplete DOYs.

10 Algorithm 7: Repair Failed DOYs

Only the incomplete DOYs are retrained. If `Q` is small, this is much cheaper than retraining all 366 DOYs.

```
Algorithm RERUN-FAILED-DOYS
Input:
  bucket-year training dataset
  bucketed climatology
  incomplete DOYs Q
  Local DOY half-window h

Output:
  patch coefficient dataset

1  for each bucket b in 0..B-1 pardo do
```

```
2      train_b <- read all bucket-year training rows for bucket b
3      clim_b <- read climatology rows for bucket b
4      for each ID i in bucket b pardo do
5          recompute anomalies and lags for ID i
6          for each failed day d in Q pardo do
7              fit weighted local regression for (ID i, doy d)
8              write patch coefficients for (ID i, doy d)
9          end for
10     end for
11 end for
12 return patch coefficient dataset
```

Complexity:

$$W_{repair} = O(NQYh)$$

$$T_p = O(NQYh/p + \log(Yh))$$

If `Q << D`, repair is much faster than full training.

11 Algorithm 8: Patch Coefficients

```
Algorithm PATCH-COEFFICIENTS
Input:
  base coefficient dataset
  patch coefficient dataset
  incomplete DOYs Q

Output:
  corrected coefficient dataset

1  for each bucket b in 0..B-1 pardo do
2      base_b <- read coefficient rows for bucket b
3      patch_b <- read patch rows for bucket b
4      base_keep <- rows in base_b whose doy is not in Q
5      corrected_b <- union(base_keep, patch_b)
6      remove duplicate (ID, doy), keeping the patch row
7      sort corrected_b by (ID, doy)
8      write corrected coefficients for bucket b
9  end for
10 return corrected coefficient dataset
```

Complexity:

$$W_{patch} = O(ND + NQ)$$

$$T_p = O((ND + NQ)/p)$$

Peak memory per bucket:

$$S_{patch-bucket} = O(ND/B + NQ/B)$$

12 Algorithm 9: Optional Forecasting

Forecasting is parallel across IDs, but recursive through time for each ID. That means the horizon loop cannot be fully parallelized within one ID because day `t` depends on previous lagged anomaly values.

```
Algorithm FORECAST-TD
Input:
  coefficients per (ID, doy)
  climatology per (ID, doy)
  future TMIN values
  forecast horizon H

Output:
  predicted TD for each (ID, date)

1  for each ID i pardo do
2      initialize lagged TD/TMIN anomalies from observed history
3      for forecast day t from 1 to H do
4          d <- day_of_year(date_t)
5          beta <- coefficients(i, d)
6          x <- current TMIN anomaly and lagged anomaly features
7          TD_anom_hat <- beta dot x
8          TD_hat <- TD_clim(i, d) + TD_anom_hat
9          update lagged anomalies with TD_anom_hat and TMIN_anom(t)
10         write prediction (ID i, date_t, TD_hat)
11     end for
12 end for
13 return predictions
```

Complexity:

$$W_{forecast} = O(NH)$$

Parallel time:

$$T_p = O(NH/p + H)$$

The `+ H` term remains because the forecast for one ID is recursive over time.

13 Old Design vs Bucket-Year Design

The original Colab-style design trained one task per `ID`. Each task discovered, opened, and filtered the same monthly parquet files.

Let `S = 12Y` be the number of monthly source files per variable.

Old I/O task overhead:

$$O(NS)$$

For `N = 300,000` and `Y = 40`, this means roughly:

$$300,000 \times 480 = 144,000,000$$

file-touch attempts per variable family before considering actual row reads.

The bucket-year design changes the I/O pattern:

- Source files are scanned once during preprocessing.
- Training reads compact bucket-year files.
- Workers write disjoint bucket outputs.
- No large driver-side concatenation is required.

New source scan overhead:

$$O(S)$$

New model training work:

$$O(NDYh)$$

So the redesign does not remove the statistical work of fitting many local regressions, but it removes the repeated source-file scanning that made the previous implementation slow and fragile.

14 Summary Complexity Table

Phase	Sequential work	Ideal parallel time with `p` processors	Peak memory target
Build climatology	`O(R)`	`O(R/p + log R)`	`O(ND)` output
Build bucket-year dataset	`O(R)` or `O(R log(ND/B))` with sorting	`O(R/p + log(ND/B))`	`O(ND/B)` per compaction
Shard climatology	`O(ND)`	`O(ND/p + log B)`	`O(ND/B)` streaming
Train coefficients	`O(NDYh)`	`O(NDYh/p + log(Yh))`	`O(NT/B + ND/B)` per bucket
Check completeness	`O(ND)`	`O(ND/p + log(ND))`	`O(ND)` or bucket streaming
Repair `Q` failed DOYs	`O(NQYh)`	`O(NQYh/p + log(Yh))`	same as training bucket
Patch coefficients	`O(ND + NQ)`	`O((ND + NQ)/p)`	`O(ND/B + NQ/B)` per bucket
Forecast horizon `H`	`O(NH)`	`O(NH/p + H)`	`O(NH/B)` per output bucket

15 Overall Complexity

The full pipeline is a sequence of parallel phases, so the total work is the sum of the work of each phase. The dominant terms are:

- bucket-year preprocessing,
- full coefficient training,
- optional failed-DOY repair,
- optional forecasting.

Using `R = NYD`, the total sequential work for preprocessing plus full training without repair and without forecasting is:

$$W_{total} = O\left(R \log\left(\frac{ND}{B}\right) + NDYh\right)$$

Equivalently:

$$W_{total} = O\left(NYD \log\left(\frac{ND}{B}\right) + NDYh\right)$$

If sorting cost inside bucket-year files is ignored or treated as an implementation detail, the simpler expression is:

$$W_{total} = O(NYD + NDYh) = O(NDYh)$$

because `h` is the main local-regression multiplier.

If repair is needed for `Q` failed DOYs, add:

$$W_{repair-total} = O(NQYh)$$

So full training plus repair is:

$$W_{total+repair} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q)\right)$$

If forecasting is also executed for horizon `H`, add:

$$W_{forecast-total} = O(NH)$$

Therefore the most complete end-to-end expression is:

$$W_{end-to-end} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q) + NH\right)$$

For parallel execution, the phases are sequentially dependent, but each phase uses `pardo` internally. With `p` processors and `B` bucket tasks, the practical parallel time is:

$$T_p = O\left(\frac{NYD \log(ND/B)}{p} + \frac{NDYh}{\min(p, B)} + \frac{NQYh}{\min(p, B)} + \frac{NH}{p} + H + \log(NYD)\right)$$

The `+ H` term remains because forecasting is recursive over time for each ID. If forecasting is not executed, remove `NH/p + H`. If no failed DOYs exist, remove the `NQYh / min(p,B)` term.

For this project, the practical bottleneck after the bucket-year redesign is:

$$O(NDYh)$$

That is the cost of fitting local regressions for many IDs and all days of year. The redesign mainly reduces repeated I/O from the old Colab-style approach.

16 HPC Interpretation

The total work complexity does not depend on `p` because it measures the total number of operations required by the algorithm. The same joins, anomaly calculations, local regressions, repairs, and forecasts must be computed whether the job runs on one processor or on a university high performance computing cluster.

On an HPC system, the relevant measure for execution time is the parallel runtime:

$$T_p = O\left(\frac{W}{p_{eff}} + \text{I/O overhead} + \text{scheduling overhead} + \text{sequential dependencies}\right)$$

where `p_eff` is the effective number of processors that the algorithm can use. It can be smaller than the physical number of available cores because of bucket count, memory bandwidth, file-system bandwidth, scheduler overhead, and dependencies between pipeline stages.

For the current bucket-based training design:

$$p_{eff} \leq B$$

This is because the main training stage creates one large parallel task per bucket. If `B = 1024`, the training stage can naturally expose up to about 1024 parallel bucket tasks. If the HPC provides fewer than 1024 worker slots, the cluster can be used efficiently. If the HPC provides far more than 1024 worker slots, the extra processors will not reduce training time much unless we increase the number of buckets or add another level of parallelism inside each bucket, for example by splitting work by `ID` or by groups of `doy`.

A practical rule for an HPC deployment is:

$$B \geq 4p$$

where `p` is the number of worker slots planned for the run. This gives the scheduler enough bucket tasks to balance slow and fast workers. However, `B` should not be made arbitrarily large because too many small buckets create many small files and increase scheduling overhead.

Example starting points:

Planned worker slots	Suggested bucket count
64	512 or 1024
128	1024 or 2048
256	2048 or 4096
512	4096 or 8192

Therefore, for a university HPC, the project should be evaluated using both:

- total work, which describes the size of the scientific computation; and
- parallel runtime, which describes how much wall-clock time can be reduced by using the cluster.

The expected speedup is strong while `p_eff` increases, but it eventually saturates because of I/O bandwidth, scheduler overhead, bucket count, and the recursive forecast dependency over the horizon `H`.

17 Practical Notes for This Machine

With about 300k IDs, 40 years, and 32 logical CPU threads, the best strategy is to keep the bucket count much larger than the number of workers. For example, `B = 1024` gives roughly `N/B approx 293` IDs per bucket, which keeps each bucket task large enough to amortize overhead but small enough to fit in memory.

The GPU is not expected to help much in the current design because the main computation is many small weighted least squares fits plus parquet I/O. The largest speedups should come from:

- avoiding repeated parquet scans,
- using bucket-year files,
- keeping worker memory bounded,
- parallelizing by bucket,
- and repairing only failed DOYs instead of repeating all training.